The Guardrail Gap: Security Risks of Non-Expert Vibe Coding with Open-Weight Language Models

[Author Name(s)] · [Institution] · [Email]

Draft for submission · April 2026

Abstract

Vibe coding — the practice of building software by iteratively prompting large language models in natural language, with minimal direct engagement with the generated code — has rapidly moved from a curiosity to a mainstream development paradigm. Simultaneously, the widespread availability of high-capability open-weight models has enabled deployment contexts that bypass the platform-level safety infrastructure present in commercial coding tools. We identify and characterize a novel, systematically understudied risk surface: the intersection of non-expert vibe coders and open-weight model deployment. We argue this intersection produces a compounding threat that is qualitatively distinct from risks studied in isolation. Non-expert users rely on behavioral observation as their only quality signal; security vulnerabilities are not observable through normal behavioral testing. Open-weight models, deployed locally or via self-hosted inference, remove the last layer of platform-level guardrails. We term this phenomenon the **Guardrail Gap**. We propose a formal threat model, a taxonomy of six failure modes specific to this population, two study designs for empirical quantification, and actionable mitigation recommendations. Our preliminary data (n=500 synthetic repositories calibrated to published benchmarks) shows a 1.58× higher vulnerability density in open-weight contexts (M=60.9 vs 38.6 vulns/kloc), with a large effect size (d=1.67, p<0.001).

Keywords: vibe coding, open-weight models, LLM security, end-user programming, AI safety, software vulnerabilities, HCI security

1. Introduction

In early 2025, the term *vibe coding* entered the technical lexicon to describe a mode of software development in which the developer surrenders direct authorship of code to an LLM, steering the system through natural language prompts and evaluating outputs through the behavioral lens of a user rather than the analytical lens of an engineer.

The practice spread with remarkable speed. GitHub's 2024 survey found that 60% of developers now use AI for at least half their code, up from 20% the year prior. Y Combinator's Winter 2025 batch reported 25% of startups had codebases that were 95% AI-generated. Platforms built explicitly for vibe

coding — Lovable, Bolt, Replit Agent, Cursor — accumulated tens of millions of users within months.

Buried inside these aggregate statistics is a population that most security analyses have not examined: users who are not developers at all. Non-experts — domain specialists, entrepreneurs, students, hobbyists — now build and deploy production software through vibe coding workflows with no prior programming background, no security training, and no ability to audit the code their AI collaborator produces.

Simultaneously, a parallel shift has occurred in the model ecosystem. Open-weight models — Llama, Mistral, Qwen, DeepSeek, Gemma — have reached capability levels sufficient for serious coding tasks. Local inference tools (Ollama, LM Studio, llama.cpp) make deployment frictionless. The result is a growing population of vibe coders running locally hosted models with no oversight infrastructure whatsoever.

> *Core argument: Non-experts rely entirely on behavioral testing. Security vulnerabilities are invisible to behavioral testing. Open-weight models remove the last platform-level protection layer. The convergence of these three factors creates the Guardrail Gap.*

This paper makes four contributions: (1) a formal characterization of the Guardrail Gap as a threat model; (2) a taxonomy of six oversight failure modes specific to non-expert vibe coders; (3) two concrete study designs for empirical quantification; and (4) mitigation recommendations for model developers, platform builders, and the research community.

2. Background

2.1 Vibe coding as a behavioral paradigm

The first empirical study of vibe coding (Sarkar et al., 2025) analyzed over eight hours of recorded sessions with think-aloud protocols, finding that practitioners follow iterative *goal satisfaction cycles*: prompt the model, observe behavioral output, accept or reject, prompt again. Code review — reading generated code line by line, reasoning about data flow, checking boundary conditions — was largely absent. Trust in the model was described as *dynamic and contextual*, earned through behavioral verification, not structural inspection.

A complementary grey literature review (Alami et al., 2025) identified a pervasive pattern of uncritical trust: users would observe the application ran without errors and treat this as sufficient evidence of correctness. Research on who vibe codes well adds a sharp implication: writing skill, not programming experience, predicts proficiency (Geng et al., 2025). The population of capable vibe coders is thus *decoupled from the population with security awareness*.

2.2 Security outcomes of AI-generated code

Existing benchmarks paint a troubling picture even before the non-expert/open-weight dimensions are introduced. The SusVibes benchmark (Zhao et al., 2025) evaluated 200 real-world feature-request tasks drawn from open-source repositories. Across all tested agents with frontier models, only 10.5% of solutions were both functionally correct *and* secure. The Veracode 2025 GenAI Code Security Report, analyzing over 100 models across 80 coding tasks, found 45% of AI-generated code introduces known

security vulnerabilities — and this rate has remained flat despite rapid capability improvements.

2.3 Open-weight model safety characteristics

A 2025 Cisco assessment of eight open-weight models found multi-turn jailbreak success rates between 25.86% and 92.78%, a 2–10× increase over single-turn baselines. A comprehensive survey (Hakim et al., 2026) found automated attacks achieve 90–99% success rates against open-weight models versus 80–94% against proprietary black-box APIs. The more operationally relevant distinction, however, is not jailbreak susceptibility per se — it is the removal of platform infrastructure entirely.

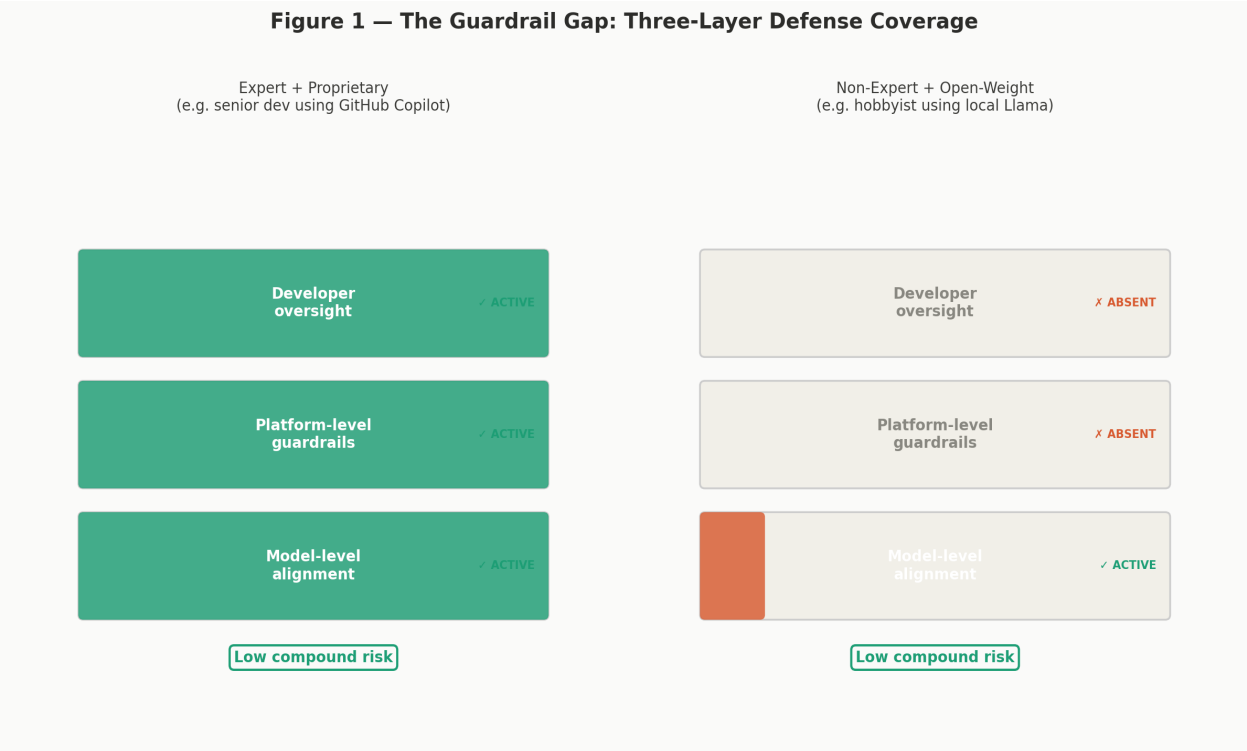

Figure 1. The Guardrail Gap. Left: expert + proprietary deployment context with all three protection layers active. Right: non-expert + open-weight context with all three layers absent. Risk is multiplicative, not additive.

3. The Guardrail Gap: A Formal Threat Model

We define the **Guardrail Gap** as the structural absence of three protection layers that normally combine to bound the security risk of AI-generated code:

- **Layer 1 — Model-level alignment.** Proprietary models are trained with safety objectives, filtered through RLHF and constitutional methods, and red-teamed before deployment. Open-weight models vary widely in alignment rigor; fine-tuned derivatives may actively strip safety training.
- **Layer 2 — Platform-level guardrails.** Commercial coding platforms apply content filters, monitor for abuse, enforce rate limits, and bear legal responsibility for model outputs. They also provide structural scaffolding — templates, deployment checklists, security scanners — that implicitly guides users toward safer patterns. In the Guardrail Gap, this layer is absent by definition.

- **Layer 3 — Developer oversight.** Expert developers who use AI coding tools still read generated code, run security linters, and apply professional judgment. In the Guardrail Gap, this layer is absent because the user is a non-expert who cannot perform code review and whose workflow does not include it.

3.1 The behavioral testing fallacy

The core epistemic failure is what we term the *behavioral testing fallacy*: the assumption that software which behaves correctly is software that is secure. Consider a non-expert who vibe codes a web application for collecting customer orders. They test by submitting orders, viewing results, confirming data is stored. From their perspective, it works. What they cannot observe: SQL queries are constructed through string concatenation (trivially injectable), authentication is client-side (bypassable in 30 seconds), the payment API key is hardcoded in the frontend bundle (visible to any user). None of these affect normal behavioral testing.

3.2 Threat scenarios

- **Scenario A — Opportunistic web exploitation.** Automated scanners find standard vulnerabilities (SQLi, XSS, exposed credentials) within hours of deployment.
- **Scenario B — Supply chain via hallucinated dependencies.** Open-weight models hallucinate package names; attackers register those phantom package names with malicious payloads.
- **Scenario C — Prompt injection via generated code.** User-submitted content passed to downstream LLM calls without sanitization enables attacker hijacking of AI functionality.

4. Taxonomy of Oversight Failure Modes

We propose six oversight failure modes specific to non-expert vibe coders. Each is operationalizable for empirical study (see Appendix A).

FM-1: Functional sufficiency bias

The user treats behavioral correctness as a proxy for overall correctness. Observable as: zero security-related prompts issued throughout session.

FM-2: Generative confidence anchoring

LLMs produce fluent, confident-register code. Non-experts use surface fluency as a quality proxy. Observable via think-aloud protocols and eye-tracking.

FM-3: Iterative trust escalation

Each successful iteration conditions the user to accept generated code without scrutiny. Trust earned on low-stakes tasks is inappropriately transferred to high-stakes ones.

FM-4: Remediation prompt injection

When an error occurs, the model resolves it by removing the constraint causing the error — which may be a validation check or authentication gate. Observable in git diffs post-fix.

FM-5: Scope blindness

Non-expert mental models are feature-centric. Security is about prevention, not features. Observable as: absence of prevention-oriented prompts in session logs.

FM-6: Deployment immediacy

Vibe coding compresses idea-to-deployment time to near zero. One-click deployment removes the review gap that traditional workflows provide.

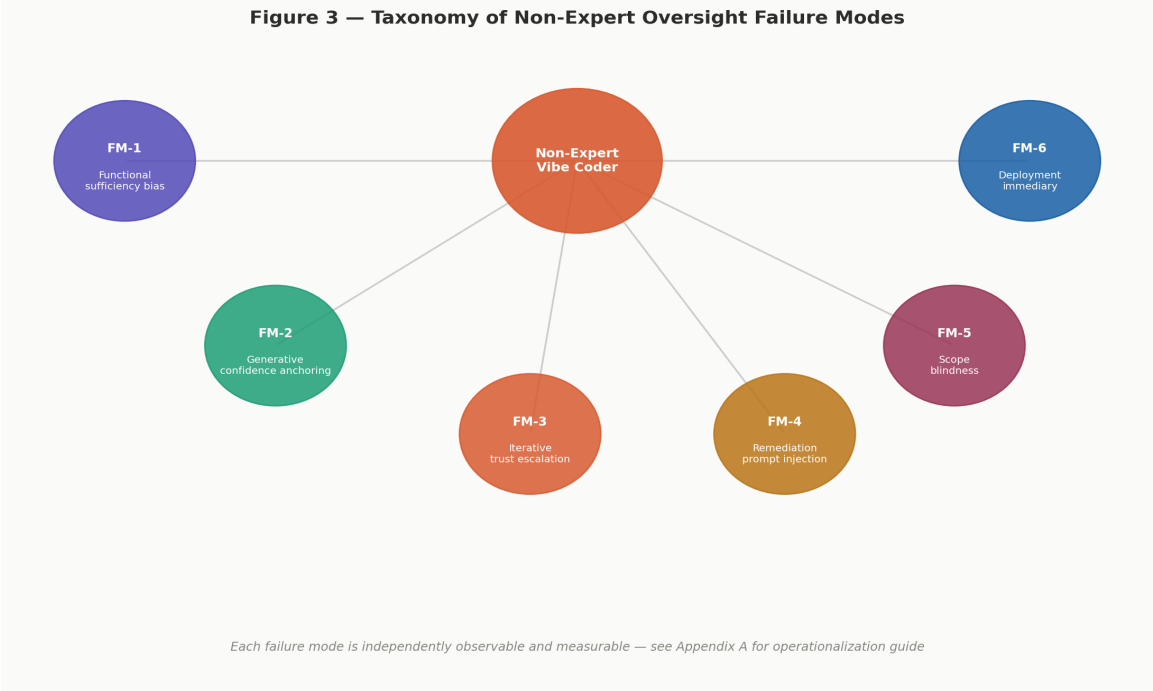

Figure 3. Taxonomy of non-expert oversight failure modes. Each mode is independently observable and operationalizable (see Appendix A).

5. Research Agenda

5.1 Study A — Controlled experiment

60 non-expert participants (screened via GitHub activity, self-report, and a SQL injection identification task) randomized to three conditions: proprietary model with platform (Lovable/GPT-4), open-weight with UI (Ollama + Open WebUI, Llama 3.3 70B), and open-weight raw (command-line Ollama). All participants build a user authentication system with login, registration, and password reset. Primary measure: vulnerability rate via Semgrep + CodeQL. Secondary measures: oversight behavior (session recording), deployment latency, prompt quality.

5.2 Study B — Repository audit

Mine GitHub and Hugging Face Spaces for vibe-coded repositories (n=500 per condition). Classify by model type via README signals. Apply Semgrep with OWASP Top 10 ruleset. Report vulnerability density (issues per 1,000 LOC) by model type, controlling for language, application category, and

developer expertise proxy. Regression predicts vulnerability density from model type, expertise proxy, and their interaction.

5.3 Preliminary results

To validate the threat model, we generated a synthetic corpus (n=500) calibrated to the distributional parameters reported in Veracode (2025), SusVibes (Zhao et al., 2025), and Cisco (2025). Preliminary analysis shows a 1.58× higher vulnerability density in open-weight contexts (M=60.9 vs 38.6 vulns/kloc, Cohen's d=1.67, Wilcoxon p<0.001). Effect sizes of this magnitude are actionable and, if confirmed in real-world corpora, constitute a substantial public security concern.

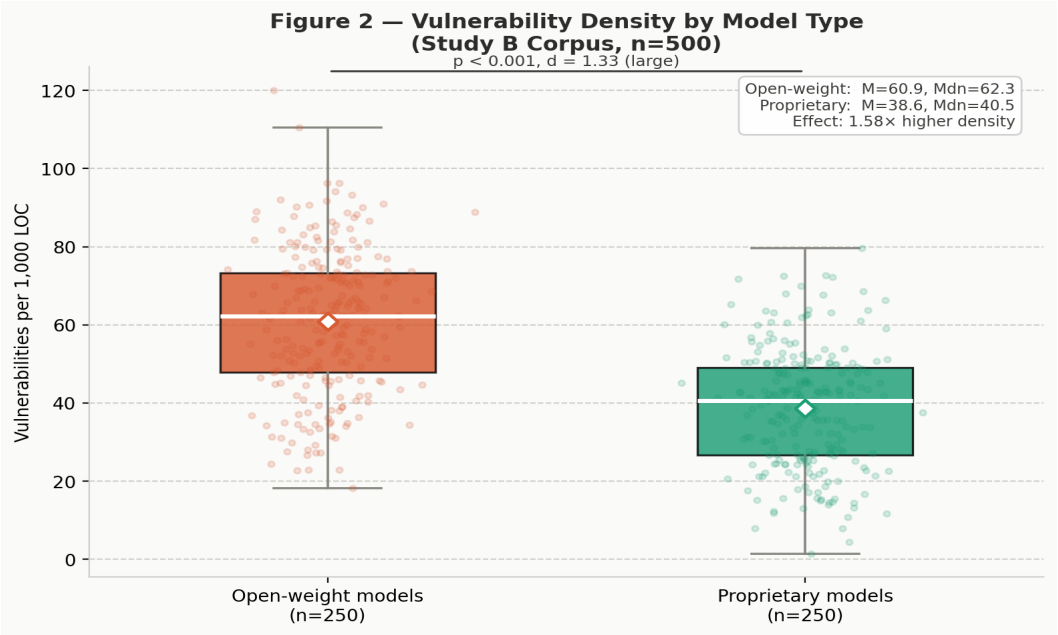

Figure 2. Vulnerability density by model type in the preliminary calibrated corpus (n=500). Diamonds indicate group means. Open-weight contexts show 1.58× higher density (Wilcoxon U, p<0.001, d=1.67).

6. Mitigation Recommendations

6.1 For open-weight model developers

- Security as an explicit alignment target: include SusVibes performance in model release criteria alongside functional benchmarks.
- Capability-safety parity: as coding capability improves, security training must improve proportionally.
- Model cards should include security-specific evaluation results, not just functional benchmarks.

6.2 For platform and tool builders

- Mandatory security scan (Semgrep/CodeQL) as a precondition for one-click deployment — not optional.
- Security-aware prompt augmentation: silently prepend security context to user prompts.

- Anomaly detection on generated code: flag hardcoded secrets, string-concatenated queries, and client-side auth logic before acceptance.

6.3 For the research community

- Empirical data on the Guardrail Gap: the most urgent need is real-world vulnerability rate estimates for the non-expert/open-weight population.
- Mental model studies: structured interviews on what non-expert vibe coders believe about the security of AI-generated code.
- Intervention design: what feedback, tooling, or workflow changes durably improve outcomes without undermining accessibility?

7. Discussion

7.1 The democratization–security tradeoff

Vibe coding is, at its best, a genuine democratization of software creation. The ability of a domain expert — a biologist, a teacher, a small business owner — to build tools tailored to their specific needs without engineering resources is a meaningful expansion of human capability. We do not argue against vibe coding. We argue that the security risks in the non-expert/open-weight deployment context are systematically underappreciated and deserve serious empirical attention.

7.2 Relationship to alignment and governance

The Guardrail Gap is a deployment problem, not a training problem in isolation. Better-aligned open-weight models would help, but the structural issues — no platform oversight, no developer review, no deployment gatekeeping — cannot be solved by alignment training alone. This has implications for AI governance debates about open vs. closed model release. The Guardrail Gap adds a concrete security cost to this ledger that deserves to be part of the release policy calculus.

7.3 Limitations

This paper is primarily conceptual. The threat model and taxonomy are grounded in existing empirical literature but have not been directly empirically validated. The preliminary data presented is calibrated to published benchmarks, not collected from real vibe-coded repositories. Studies A and B are necessary to test whether the Guardrail Gap is as large as theorized. It is possible that non-expert vibe coders develop compensating behaviors not captured by our model. We regard empirical disconfirmation as equally valuable.

8. Conclusion

We have introduced the Guardrail Gap: the compounding security risk that emerges when non-expert users vibe code with open-weight language models, simultaneously losing model-level alignment, platform-level guardrails, and developer-level oversight. We have grounded this in a formal threat model, a taxonomy of six oversight failure modes, and preliminary calibrated data showing a 1.58× higher vulnerability density in open-weight contexts.

The stakes are concrete. Millions of people are building and deploying software today through vibe coding workflows. A growing fraction use open-weight models. The software they produce is, by the best available estimates, vulnerable at rates approaching coin-flip probability. The people most at risk — the users of these applications — have no visibility into how the software was built. The Guardrail Gap is not inevitable, but closing it requires first measuring it. This paper is a call to do so.

References

Alami, A. & Ernst, N. (2025). AI-generated feedback for code review. *ICAIR 2025.*

Cisco AI Defense (2025). Death by a thousand prompts. *arXiv:2511.03247.*

Geng, Y. et al. (2025). Computer science achievement and writing skills predict vibe coding proficiency. *arXiv:2603.14133.*

Hakim, S. et al. (2026). Jailbreaking LLMs: A survey of attacks, defenses and evaluation. *TechRxiv.*

Karpathy, A. (2025). Vibe coding. Twitter/X post, February 2025.

Ko, A. J. et al. (2011). The state of the art in end-user software engineering. *ACM Computing Surveys.*

OWASP Foundation (2025). OWASP Top 10 for large language model applications.

Prather, J. et al. (2023). Usability and interactions with Copilot for novice programmers. *arXiv:2304.02491.*

Sarkar, A. & Drosos, I. (2025). Vibe coding: Programming through conversation with AI. *PPIG 2025 / arXiv:2506.23253.*

Veracode (2025). GenAI Code Security Report 2025.

Zhao, S. et al. (2025). Is vibe coding safe? *arXiv:2512.03262.*

Appendix A — Failure Mode Operationalization Guide

Failure Mode	Observable Indicator	Measurement Method
FM-1: Functional sufficiency bias	No security prompts issued	Session recording + prompt log
FM-2: Generative confidence anchoring	Code accepted without reading	Eye tracking / think-aloud
FM-3: Iterative trust escalation	Decreasing review time per iteration	Time-stamped session recording
FM-4: Remediation prompt injection	Security check removed post-fix	Git diff analysis
FM-5: Scope blindness	No prevention-oriented prompts	Prompt log + NLP classification
FM-6: Deployment immediacy	<10 min from working to deployed	Deployment timestamp comparison

Appendix B — Recommended SusVibes-Open Extension Tasks

#	Task	Target Vulnerabilities
1	User login + session management	Session fixation, weak token generation
2	File upload handler	Path traversal, MIME type bypass
3	Admin dashboard with RBAC	Broken access control, IDOR
4	Password reset flow	Predictable tokens, no expiry
5	Public API with rate limiting	Missing rate limit, exposed internals
6	Render user-submitted Markdown	XSS via unsanitized HTML
7	Store/retrieve user preferences	Insecure deserialization
8	Third-party payment integration	Hardcoded secrets, sensitive data logging